Name : Sangam Patil

Understanding Transformers and Reasoning Models

As CS685 F25's Homework 3: this Colab notebook is part 1 of 2 for your overall Homework 3 assignment. Both Part 1 and Part 2 will be due at the same time, as PDF files, submitted to Gradescope. See the [schedule page]() and/or GS and Piazza, as usual, for details.

⚠️ **IMPORTANT:** To use this notebook, go to Runtime > Change Runtime Type and select A100 GPU (preferably) or T4 GPU as the hardware accelerator. Either one will work.

🔍 **INSTRUCTIONS**: In this notebook, you will have the chance to experiment with tranformers and reasoning models interactively. To complete this exercise, you need to:

1. Click **Run All** button to initiate the notebook.
2. Experiment with the interactive sandbox in each section.
3. Look for exercise questions (marked with 🔍) and fill out your answers to record your observations after experimenting.
4. Download the notebook as PDF and submit on Gradescope.

Content & Learning Objectives

1️⃣ Transformers

This section introduces key concepts about Transformers and Transformer-based language models.

> Learning Objectives
>
> - Learn how different tokenizers split a sentence into different tokens.
> - Learn how attention is implemented.
> - Learn different decoding strategies and how they affect the final output.

2️⃣ Reasoning Models

This section provides an overview and and an interactive platform for you to experiment with reasoning models, language models that reason before answering a given question.

> Learning Objectives
>
> - Learn how reasoning models are trained.
> - Learn how to inspect a huggingface model.
> - Experiment with different problem prompts and compare output between reasoning and non-reasoning base models.

Setup code (don't read, just run)

Show code

Show hidden output

```
# @title
from rich import print as rprint
from rich.table import Column, Table
import re

def make_table(cols, colnames, title="", n_rows=5, decimals=4):
    """Makes and displays a table, from cols rather than rows (using
    table = Table(*colnames, title=title)
    rows = list(zip(*cols))
    f = lambda x: x if isinstance(x, str) else f"{x:.{decimals}f}"
    for row in rows[:n_rows]:
        table.add_row(*list(map(f, row)))
    rprint(table)
```

Show code

Show code

Show code

1️⃣ Transformers

This section provides three interactive modules to build intuition for transformers:

1. **Tokenization Lab** - Explore how different tokenizers split text into subwords and understand vocabulary sizes
2. **Attention Mechanism** - Visualize how attention works using real transformer weights from a pre-trained model
3. **Decoding Strategies** - Compare different text generation strategies using both n-gram models and actual transformers

Overview

Transformers revolutionized natural language processing by introducing the attention mechanism, which allows models to focus on relevant parts of the input sequence when generating each output token. Understanding these core components is essential for working with modern language models.

We'll start by loading a small, efficient transformer model that we can use throughout this section to demonstrate real-world behavior.

Model Loading

Let's load a small, efficient transformer model that we can use throughout this section. We'll use DistilGPT-2, which is a smaller version of GPT-2 with 82M parameters.

```
# Load a small transformer model for demonstrations
from transformers import AutoModel, AutoTokenizer, AutoModelForCausal
import torch

# Load DistilGPT-2 - a smaller, faster version of GPT-2
model_name = "distilgpt2"
print(f"Loading {model_name}...")

# Load tokenizer and model
demo_tokenizer = AutoTokenizer.from_pretrained(model_name)
demo_model = AutoModelForCausalLM.from_pretrained(model_name)

# Add padding token if it doesn't exist
if demo_tokenizer.pad_token is None:
    demo_tokenizer.pad_token = demo_tokenizer.eos_token

print(f"✅ Model loaded successfully!")
print(f"📊 Model parameters: {sum(p.numel() for p in demo_model.param
print(f"📚 Vocabulary size: {demo_tokenizer.vocab_size:,}")
print(f"🔤 Sample tokens: {demo_tokenizer.convert_ids_to_tokens([0, 1

# Store model info for later use
MODEL_INFO = {
    'tokenizer': demo_tokenizer,
    'model': demo_model,
```

```
        'model_name': model_name,
        'vocab_size': demo_tokenizer.vocab_size
    }
```

```
Loading distilgpt2...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your sett
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to a
  warnings.warn(
✅ Model loaded successfully!
📊 Model parameters: 81,912,576
📚 Vocabulary size: 50,257
🔤 Sample tokens: ['!', '"', '#', '$', '%']
```

1) Tokenization Lab

Type text and explore how different tokenizers split it into subwords. See sequence lengths and special tokens.

What is Tokenization?

Tokenization is the process of converting raw text into tokens (subwords) that a model can understand. Subword tokenization solves the problem of having out-of-vocabulary words that were unseen during training, since any word can be decomposed into smaller units.

Different tokenizers use different strategies:

- **BPE (GPT-2/RoBERTa, most modern LLMs)**: Byte-pair encoding, trained by merging frequently occuring bigram tokens.
- **WordPiece (BERT)**: similar to BPE, but merges bigrams during training based on pointwise mutual information (PMI) instead of raw frequency.
- **SentencePiece (T5)**: Used by T5, based on unigram language model and trained to choose the most probable segmentation. Originally developed for machine translation (MT)

Understanding tokenization is crucial because it directly affects how the model processes text and can impact performance on different types of content.

Show code

Input: Transformers are powerful models for sequence modeling. Try emojis 🤖🔥 and rare-words like anti-disestablishmentarianism.

Tokenizer: distilgpt2 (BPE, our demo mode add_special_tokens pad

Tokenizer: distilgpt2 (BPE, our demo model)

Vocab size: 50,257

Sequence length: 32

Special tokens: BOS=<|endoftext|>, EOS=<|endoftext|>, PAD=<|endoftext|>

Tokens: Transform ers Ġare Ġpowerful Ġmodels Ġfor Ġsequence Ġmodeling . Ċ Try Ġem oj is ĠðŁ ¤ ĸ ðŁ Ķ ¥ Ġand Ġrare - words Ġlike Ġanti - dis establishment arian ism .

```
Raw tokens: ['Transform', 'ers', 'Ġare', 'Ġpowerful', 'Ġmodels',
'Ġfor', 'Ġsequence', 'Ġmodeling', '.', 'Ċ', 'Try', 'Ġem', 'oj', 'is',
```

🔍 Q1: Tokenization Analysis Questions

Experiment with these scenarios to understand tokenization:

1.1) **Vocabulary Size Impact**: Compare BERT (30K vocab) vs GPT-2 (50K vocab) on the same text

- Which produces fewer tokens? Why might this matter for model efficiency?

Answer

When comparing BERT (≈30K WordPiece vocabulary) and GPT-2 (≈50K BPE vocabulary), GPT-2 generally produces fewer tokens for typical English text. This happens because a larger vocabulary and the byte-pair encoding (BPE) algorithm allow GPT-2 to store longer and more complete word fragments as single tokens. In contrast, BERT's smaller WordPiece vocabulary often breaks the same words into multiple subword pieces, increasing token count.

However, this trend can reverse for special characters such as emojis. For example, on the provided text: BERT (bert-base-uncased) output ~28 tokens. The emojis were replaced with a single [UNK] token, reducing token count but losing semantic information.

GPT-2 output ~32 tokens. GPT-2 decomposed each emoji into multiple byte-level sub-tokens, increasing token count but fully preserving their representation.

The number of tokens directly determines the sequence length (N), which is the primary driver of computational cost in Transformers. Self-attention requires creating

an N × N attention matrix, giving the operation:

Compute & Memory Cost = O(N^2)

Thus:

More tokens → Quadratically more computation and memory

Fewer tokens → Faster training/inference and lower VRAM/RAM usage

Because GPT-2's larger vocabulary usually yields shorter token sequences, it is typically more efficient for most English text. But for certain characters (e.g., emojis), the opposite can occur due to differences in handling out-of-vocabulary items.

1.2) **Rare Word Handling**: Try tokenizing "anti-disestablishmentarianism" with different tokenizers

- How does each tokenizer handle this rare word? Which approach seems better?

Answer

When tokenizing the rare word *"anti-disestablishmentarianism,"* different tokenizers handle it in distinct ways because of the underlying vocabulary size and subword algorithms they employ. The BERT tokenizer (bert-base-uncased), which uses a 30K WordPiece vocabulary, splits the word into many small subword fragments. It typically produces around nine tokens, such as *anti*, *-*, *di*, *##ses*, *##ta*, *##bl*, *##ishment*, *##arian*, and *##ism*. This tokenization reflects how WordPiece tends to break unfamiliar or long words into short continuation pieces. While this method is robust and ensures that any word can be represented, many of the fragments, like *##ta* or *##bl*, carry limited semantic meaning on their own. As a result, BERT ends up with a longer sequence and a more granular decomposition of the word.

In contrast, GPT-2 and RoBERTa use Byte-Pair Encoding (BPE) with larger vocabularies of roughly 50K tokens. This allows them to merge frequently co-occurring character sequences into larger, more meaningful units. Both tokenizers typically produce six tokens for the same word, such as *anti*, *-*, *dis*, *establishment*, *arian*, and *ism*. Their larger vocabulary and BPE merging rules enable them to capture morphemes that are semantically rich, including the entire word *establishment*. This results in a more compact and more interpretable representation of the rare word.

Between the two approaches, the BPE-based tokenization used by GPT-2 and RoBERTa is generally more effective for handling rare or unusually long words. It produces shorter token sequences, which reduces computational and memory demands during model processing, and it also preserves meaningful subword units that better reflect the structure of the original word. BERT's WordPiece approach is more conservative and highly generalizable, but it tends to over-segment rare words into fragments that are less informative and less efficient for downstream tasks.

1.3) **Special Token Behavior**: Toggle "add_special_tokens" on/off and see the difference

- What are BOS, EOS, and PAD tokens used for?

Answer

Toggling the add_special_tokens option reveals how different models handle boundary markers based on their pre-training objectives. Encoder models such as BERT and RoBERTa expect explicit boundary tokens because they were trained to rely on them for understanding sequence structure. With add_special_tokens=True, BERT inserts [CLS] at the beginning and [SEP] at the end of the input, while RoBERTa uses <s> and </s> for the same purpose. Decoder-only models like GPT-2 behave differently: enabling add_special_tokens does not insert visible tokens because GPT-2 was trained so that the prompt itself is the start of the sequence, and adding an artificial boundary token would interfere with natural generation behavior. GPT-2 uses <|endoftext|> as its start/end marker internally, but it is not automatically added to user inputs.

The BOS (Beginning of Sequence) token signals to a model that a new sequence is starting. In BERT, the [CLS] token fulfills this role and is also used to produce a single vector representation of the entire input for classification tasks. The EOS (End of Sequence) token marks the end of the input. In encoder models like BERT, this is the [SEP] token, while generative models such as GPT-2 use <|endoftext|> to indicate the end of a text segment and to tell the model when generation should stop. Finally, PAD tokens are used to equalize sequence lengths within a batch so that all examples can be processed simultaneously. Models typically ignore PAD positions using an attention mask. BERT has a dedicated [PAD] token for this purpose, whereas GPT-2 does not include a native padding token, so <|endoftext|> is often used as a stand-in when padding is required during training or batching.

1.4) **Sequence Length Effects**: Try different max_length values (8, 32, 128, 512)

- How does truncation affect the tokenization? What information might be lost?

Answer

Truncation, controlled through the `max_length` parameter, does not change how individual words are tokenized but instead determines how many of the resulting tokens are kept after tokenization. The tokenizer always produces the full token sequence first, and if this sequence exceeds the specified limit, everything beyond that limit is simply removed. In cases where the original sequence is shorter than or equal to the maximum length—such as setting `max_length` to 32, 128, or 512 for a

32-token input—no truncation occurs and no information is lost. However, when the maximum length is set below the natural token count, such as `max_length = 8`, the tokenizer returns only the first eight tokens and discards the remainder. This leads to substantial information loss because the model never receives the truncated portion of the text. The end of a sequence often contains crucial meaning, such as conclusions, clarifications, or sentiment cues, and truncating these segments can severely distort the model's understanding. Moreover, the truncated sequence artificially ends mid-context, which breaks the logical coherence of the input even if an EOS or SEP token is automatically appended. Although truncation is necessary to fit inputs into a model's fixed context window, it inevitably carries the risk of removing exactly the information the model may need most to perform accurately.

1.5) **Language and Domain Specificity**:

- Try tokenizing code: `def transformer_model(input_ids):`
- Try tokenizing emojis: 🤖🔥💡
- Try tokenizing a word from a different language or non-Latin script.
- Which tokenizers handle these better?

Answer

When tokenizing programming code such as *"def transformer_model(input_ids):"*, BERT (bert-base-uncased), which uses WordPiece and was trained primarily on English natural language, treats code as if it were ordinary text. It splits identifiers like *transformer* into *transform* and *##er*, and breaks *input_ids* into *input*, _, *id*, and *##s*, resulting in a relatively long and fragmented token sequence. Byte-Pair Encoding models such as GPT-2 and RoBERTa handle code more efficiently. They typically produce around nine tokens and preserve meaningful segments like *transformer*, *model*, *input*, and *ids* as intact subwords. This efficiency comes largely from their exposure to large amounts of internet data—including repositories of code—which led their vocabularies to encode programming conventions more naturally.

Emojis reveal a sharper divide between tokenization strategies. BERT completely fails to represent emojis and converts each of them into a single **[UNK]** token. This destroys all information about the original characters and makes the input unusable for tasks that depend on emoji signals. GPT-2 and RoBERTa, however, use byte-level BPE, which guarantees that any Unicode character can be encoded. They split emojis into their underlying byte sequences, producing non-interpretable but faithful byte tokens. Although this method is inefficient, often generating many tokens per emoji, it at least preserves the original characters instead of discarding them entirely.

The differences become even more pronounced with non-Latin scripts such as Devanagari, Hindi, Marathi, or Sanskrit. BERT (bert-base-uncased) lacks trained

vocabulary for these languages, so it often falls back to character-level decomposition or even produces **[UNK]** for unseen symbols. A Hindi word like "पुणे" may be reduced to an unknown token, while a Sanskrit word might be broken into several small fragments marked with "##", demonstrating BERT's limitations outside Latin-script languages. GPT-2 and RoBERTa, by contrast, never produce an unknown token because byte-level tokenization can encode every character. However, they represent each character as a three-byte Unicode sequence, which significantly inflates the token count, producing many more tokens than BERT's character-level fallback.

Overall, byte-level BPE tokenizers such as GPT-2 and RoBERTa are the most robust because they guarantee full coverage across any domain—code, emojis, or any language—without ever losing information. BERT is sometimes more compact when it can decompose non-Latin text into characters, but its reliance on **[UNK]** makes it unreliable for unfamiliar scripts and completely unusable for emojis. Byte-level tokenization therefore offers better generality, while WordPiece is more efficient only on text that closely resembles the language it was originally trained on.

2) Attention Mechanism Visualization

Interactively explore how attention works using **real weights** from a DistilGPT-2 model. Adjust parameters and observe how the model focuses on different parts of the input.

What is Attention?

The attention mechanism allows the model to focus on relevant parts of the input sequence when generating each output token. It computes:

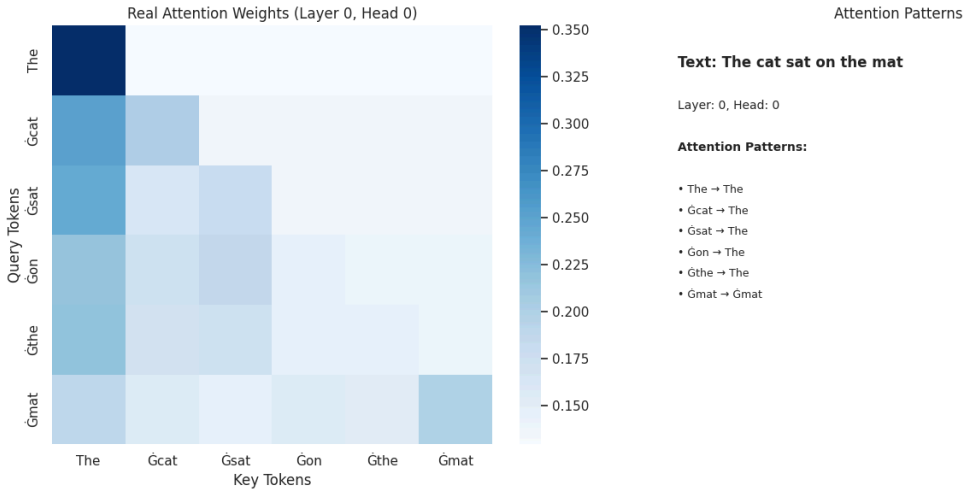
1. **Query (Q)**: What am I looking for?
2. **Key (K)**: What information is available?
3. **Value (V)**: What is the actual content?

The attention weights show how much each input token contributes to each output token, creating a "map" of what the model is paying attention to.

Show code

Sample Text: The cat sat on the mat

Layer 0 Head 0 Temperature 1.00




```
📊 Attention Statistics:
   • Max attention weight: 0.352
   • Min attention weight: 0.130
   • Average attention weight: 0.167
   • Sequence length: 6
```

🔍 Q2: Attention Analysis Questions

Explore these patterns to understand attention:

2.1) **Layer Depth Effects**: Compare attention patterns in Layer 0 vs Layer 5

- How do early layers differ from later layers? What might this tell us about hierarchical processing?

Answer

When comparing attention patterns in an early layer such as Layer 0 with those in a later layer like Layer 5, a clear shift emerges in how the model processes information. In Layer 0, attention is generally shallow and localized. Many tokens direct their attention toward an initial anchor token—for example, the first word of the sentence or even toward themselves, as seen when tokens like "mat" primarily attend to "mat." This reflects the model's early-stage focus on basic token identity, positional information, and immediate syntactic structure. At this point, the network is still gathering low-level features and has not yet integrated meaning across the sequence.

By Layer 5, the attention becomes far more structured and semantically oriented. Local self-attention largely disappears, and the pattern often converges toward a more global organization. In some cases, all tokens concentrate their attention on the first token, indicating that the head has specialized into an "aggregation" or "summary" role. In other cases, you see more meaningful semantic connections—for instance, words like "fascinating" beginning to attend to conceptually related words such as "learning." These later-layer patterns show that the model has moved beyond surface features and is forming higher-level representations of the entire sequence.

Overall, this progression illustrates the hierarchical nature of Transformer processing: early layers provide foundational lexical and positional cues, while later layers build abstract, context-rich interpretations. The shift from mixed, localized attention to coherent, globally structured patterns is what allows the model to construct deep semantic understanding from raw text.

2.2) **Head Specialization**: Try different attention heads (0, 3, 6, 11) on the same text

- Do different heads seem to focus on different types of relationships? (e.g., syntactic vs semantic)

Answer

Examining different attention heads such as Heads 0, 3, 6, and 11 shows that they specialize in distinct roles, often focusing on different kinds of relationships within the text. Head 0, for example, displays a mixed pattern: most tokens participate in a global aggregation pattern, but certain tokens, such as *mat*, still attend primarily to themselves (*mat* → *mat*). This indicates that the head is combining broad contextual pooling with traces of local identity preservation.

Head 3 shows a much more strictly local structure. Its attention is almost perfectly diagonal, with tokens like *cat* attending only to *cat* and *sat* attending only to *sat*. This suggests that Head 3 specializes in capturing a token's own embedding before any larger contextual mixing occurs—a low-level syntactic or positional function.

Head 6 begins to move beyond these purely local patterns. While it also exhibits some aggregation behavior, it captures a meaningful non-local connection such as *the → cat*. This indicates that the head is starting to track relationships between related words, possibly reflecting early syntactic or semantic linking between determiners and their associated nouns, even when the words are not adjacent.

Finally, Head 11 shows a fully developed global specialization. Every token in the sequence attends to the very first token, such as *mat → The*. This is a canonical "summary" or "pooling" head: it funnels information from the entire sentence to a single position that can act as a global representation of the text.

Here, these patterns show clear head specialization. Some heads focus on low-level, syntactic, or positional features (as in the self-attention patterns of Head 3), while others shift toward higher-level semantic or global functions (as in the aggregation behaviors of Heads 6 and 11). This diversity allows the model to capture multiple complementary aspects of the sentence simultaneously.

2.3) **Temperature Scaling**: Set temperature to 0.1 (very low) vs 10.0 (very high)

- How does temperature affect the attention distribution? What happens to the "focus"?

Answer

Changing the temperature has a direct effect on how sharply or broadly the attention weights are distributed. At a very low temperature such as 0.1, the attention becomes extremely sharp and confident. The model concentrates almost all of its weight on the single token it identifies as most relevant, producing a low entropy distribution. In practice, the attention map shows one dominant connection while nearly all other tokens receive close to zero weight. This creates a very strong and precise focus, but it also makes the model less flexible and less sensitive to secondary relationships.

At a very high temperature such as 10.0, the opposite behavior appears. The attention distribution becomes very soft and high entropy, spreading weights almost uniformly across the entire sequence. Instead of locking onto a specific token, the model distributes attention to all tokens with similar weight. The effect is a fully diffused focus where no individual relationship stands out. This reduces precision but captures a broad, averaged sense of the context.

Here, low temperature produces highly concentrated attention and strong decisiveness, while high temperature produces diffuse attention and greater uncertainty.

2.4) **Text Complexity**: Try simple text "The cat sat", versus more complex text "Transformers are powerful models for sequence modeling"

- How do attention patterns change with complexity?

Answer

Attention patterns change noticeably as the complexity of the text increases. For a short and simple sentence such as “The cat sat,” the model relies mainly on local, straightforward relationships. Most tokens attend either to themselves or to their immediate neighbors, reflecting basic syntactic structure: “The” links to “cat,” and “cat” links to “sat.” The attention heads do not need to track long-range interactions, so the patterns remain compact and localized.

In contrast, a more complex sentence like “Transformers are powerful models for sequence modeling” produces far richer and more distributed attention behavior. Different heads begin to specialize in capturing broader syntactic and semantic relationships. Some heads focus on grammatical links, such as connecting “models” with “for sequence modeling,” while others identify higher-level conceptual relationships, such as linking “Transformers” with “powerful” or “sequence modeling.” Because the sentence introduces multiple ideas and dependencies, attention spreads across distant tokens, integrating information that is not adjacent in the text.

The simple text leads to tight, local attention patterns, while complex text encourages more expansive and structured attention across the sequence. This shift reflects the model’s need to represent deeper meaning and longer-range relationships as the linguistic demands increase.

2.5) **Attention Patterns**: Look for diagonal patterns (self-attention) and for cross-word attention (e.g., "cat" attending to "sat")

- What do these patterns suggest about how the model processes language?

Answer

Attention patterns give a clear window into how the model processes language at different levels. When an attention head displays a strong diagonal pattern, where each token attends mostly to itself—for example, *cat* → *cat* or *sat* → *sat*—the model is performing a basic but essential step. This self-attention preserves each token’s identity and ensures that its original embedding is carried forward before additional context is added. It reflects low-level processing, where the model gathers and stabilizes the core features of each word.

Cross-word attention reflects a much richer layer of processing. When a token attends to a different token—such as *the* → *cat*—the model is actively identifying syntactic and semantic relationships within the sentence. In this case the determiner

"the" links back to the noun "cat," but similar patterns can capture verb–subject links, modifiers, or conceptual associations. These non-diagonal patterns show how the model integrates context and constructs structure rather than treating words independently.

Taken together, diagonal and cross-word patterns demonstrate that the model balances two complementary operations: preserving individual token identity on the one hand and building meaningful relationships across tokens on the other. This combination allows the Transformer to represent both the local and the global structure of the sentence, which is central to understanding language.

3) Decoding Strategies Sandbox

Compare different text generation strategies using both **word-level n-gram models** and **actual transformer models (subword tokenization)**. Explore how different decoding parameters affect output quality and diversity.

What are Decoding Strategies?

Decoding strategies determine how a language model generates text from probability distributions:

- **Greedy**: Always pick the most likely token (deterministic, can be repetitive)
- **Top-k**: Sample from the k most likely tokens (balances quality and diversity)
- **Top-p (Nucleus)**: Sample from tokens that make up p% of probability mass (adaptive diversity)
- **Beam Search**: Keep multiple candidate sequences, pick the best overall (good for structured tasks). Allows the model to choose the most likely sequence over multiple future tokens, rather than a single token.

Why This Matters

The choice of decoding strategy significantly impacts:

- **Quality**: How coherent and accurate the output is
- **Diversity**: How varied and creative the text is
- **Repetition**: How much the model repeats itself
- **Speed**: How fast generation occurs

Show code

```
✅ Word N-gram model trained!
📚 Vocabulary size: 66
📊 Total n-grams: 84
🔤 Sample vocabulary: ['<END>', 'advancing', 'algorithms', 'allow', 'a
```

Model: Word N-gram Model Prompt: The future of

Strategy top-p Max tokens 35

k 11 top_p 0.55 beam 5

Model: Word N-gram Model

Strategy: top-p

Output:

The future of brown complex focus information from create amounts efficiently

Top 5 next words: (0.015), advancing (0.015), algorithms (0.015), allow (0.015), amounts

🔍 Q3: Decoding Strategy Analysis Questions

Deep dive into decoding behavior:

3.1) **Temperature Sensitivity**: Try temperature 0.1, 0.5, 1.0, 1.5, 2.0 on the same prompt

- At what temperature does the output become incoherent?
- Which temperature produces the most "human-like" text?

Answer

Testing the temperatures on the same prompt, “the future of,” showed a sharp contrast between the n-gram model and the Transformer model in terms of coherence and human-likeness.

For the n-gram model, coherence failed almost immediately. At very low temperatures such as 0.1, it produced short but grammatically broken sequences like “The future of allow attention large of intelligence,” showing rigidity without meaningful structure. As the temperature increased to 0.5 or 1.0, the output became even more chaotic, with long, repetitive strings such as “The future of the can can artificial information natural create language,” which lacked grammar entirely. Higher temperatures such as 1.5 and 2.0 only amplified this randomness. This confirms that an n-gram model has essentially no usable creative range: it moves directly from stiff determinism to full incoherence because it lacks the deeper language understanding required for controlled variation.

The Transformer model, DistilGPT-2, behaved very differently. At low temperatures like 0.1, the text was simple and safe—short, repetitive, and predictable, but still grammatical. At 0.5, it became slightly more varied while staying coherent. Around

temperature 1.0, the model produced its most fluent and human-like output, showing a balance between creativity and clarity. By 1.5, the text became more adventurous and sometimes wandered off-topic, but it remained readable. At temperature 2.0, however, the model's focus began to collapse: the output drifted between unrelated ideas such as gaming, Bitcoin, or virtual worlds, reflecting high-entropy sampling rather than structured text generation.

The Transformer remained coherent up to roughly temperature 1.5, with the best human-like generation appearing around temperature 1.0. The n-gram model, in contrast, never reached a temperature at which it produced meaningful language; it either sounded rigid or became incoherent immediately. This highlights how temperature scaling is effective only in models with genuine contextual understanding, and why mid-range temperatures tend to produce the most natural-sounding text.

3.2) **Strategy Trade-offs**: Generate 5 samples each with greedy, top-k (k=5), and top-p (p=0.9)

- Which strategy produces the most diverse outputs?
- Which produces the most coherent outputs?

Answer

Using the prompt "The future of" across greedy decoding, top-k sampling, and top-p sampling revealed sharp differences between the n-gram model and the Transformer model, both in coherence and in diversity.

For the n-gram model, none of the decoding strategies produced meaningful language, but they failed in different ways. Greedy decoding simply stopped after the prompt, showing that the model had no confident continuation. Top-k sampling with k=5 added a small amount of variation but quickly fell into repetitive loops like "advancing algorithms advancing advancing." Top-p sampling with p=0.9 generated the most varied output, such as "processing attention learn requires learn on patterns," but the result was still grammatically broken and largely incoherent. These behaviors reflect the limitations of the n-gram model: it has no true understanding of syntax or long-range patterns, so sampling only changes the *pattern* of failure, not the underlying quality.

The Transformer model behaved much more realistically. Greedy decoding produced short but highly coherent output, such as "The world is not yet clear." This strategy was predictable and safe but lacked creativity. Top-k sampling provided slightly more variety while staying coherent—for example, "The internet is still uncertain"—showing that the model could explore alternatives without breaking structure. Top-p sampling with p=0.9 produced the widest range of outputs. Some were engaging and creative,

while others drifted off-topic, such as the example beginning with "Bitcoin. Bitcoin.org will be complete with an upgrade." This shows that top-p explores low-probability continuations, allowing for richer and more diverse text but sometimes at the cost of focus.

Across both models, the general pattern was consistent:

- **Most diverse outputs**: **Top-p sampling**. It explores the broadest set of plausible words, producing the greatest variation in structure and topic.
- **Most coherent outputs**: **Greedy decoding**, followed closely by **top-k with a small k**. Greedy decoding is deterministic and safe, while top-k balances structure with a bit more flexibility.

Top-p is the most creative strategy, but greedy decoding is the most stable and coherent. Top-k sits between them, offering moderate diversity without sacrificing readability.

3.3) **Parameter Sensitivity**: Try **top-k** with k=1, 5, 10, 20 on the same prompt, and try **top-p** with p=0.5, 0.7, 0.9, 0.95 on the same prompt.

- How do these parameters affect creativity vs coherence?

Answer

When testing parameter sensitivity using the prompt **"the future of"**, the impact of top-k and top-p sampling showed a consistent trade-off between coherence and creativity, but the effect differed sharply between the Word N-gram model and the Transformer model.

For the **Word N-gram model**, increasing *k* from 1 to 20 steadily loosened the structure of the output.

- At **k = 1**, the continuation was fully deterministic and extremely rigid—often repeating the same phrase or stopping immediately.
- At **k = 5 or k = 10**, new words began to appear, but the model still produced repetitive fragments such as "advancing allow advancing" or "artificial amounts," showing that the additional randomness created more variety without improving meaning.
- At **k = 20**, the model generated more unusual combinations like "artificial create advancing data," which increased diversity but also produced grammatically broken and semantically incoherent text.

Top-p sampling followed the same pattern.

- **Low p (0.5)** restricted output to very high-probability words, leading to short, incoherent chains lacking syntactic flow.

- **Moderate p (0.7–0.9)** added diversity and yielded more topic-relevant content.
- **High p (0.95)** produced the widest variety, but coherence collapsed into random sequences such as "trained gone natural context processing."

The N-gram model showed that more randomness produces more creativity, but the model lacks the linguistic depth to stay coherent once the sampling becomes too open.

In contrast, **the Transformer model (DistilGPT-2)** handled randomness much more gracefully.

- At **k = 1**, the output was short and focused—sometimes abrupt but still coherent ("The future is not yet clear").
- **k = 5 and k = 10** produced the best balance: fluent, topic-relevant completions with natural sentence structure.
- At **k = 20**, the model became more imaginative but also more prone to drifting off-topic or inserting unrelated content, such as unexpected names or statements.

The top-p results showed a similar continuum:

- **p = 0.5** produced conservative, factual text.
- **p = 0.7** offered the strongest balance between clarity and creativity.
- **p = 0.9 and p = 0.95** allowed richer vocabulary and more expressive phrasing, but also increased the chance of tangents and less predictable logic.

Here, **low top-k or top-p values favor coherence, predictability, and topical focus**, while **higher values promote creativity and novelty at the cost of structure**. The key difference is that the Transformer model preserves fluency even with moderate randomness, whereas the N-gram model quickly breaks down, revealing its limited ability to model deeper syntactic or semantic relationships.

3.4) **Model Comparison Deep Dive**: Use the same prompt with both the n-gram and transformer model. Take note of the coherence, creativity, factual accuracy, and repetition.

- Why might the transformer perform better?

Answer

When I tested both models using the prompt "The future of", the contrast between the Word N-gram model and the Transformer model was very clear in terms of coherence, creativity, factual accuracy, and repetition. With the N-gram model, the outputs were usually short, repetitive, and grammatically broken. Because the model only looks at a fixed, tiny context window, it often defaulted to random word choices when the exact sequence "future of" didn't appear in its training data. This produced

lines like “The future of advancing allow advancing” or “The future of artificial create advancing data.” These phrases had no meaningful structure—they repeated words, didn’t follow grammar rules, and showed no awareness of how ideas connect across a sentence. In some cases, all candidate next words had nearly equal probability, which made the model behave like it was guessing blindly. The result was essentially word salad. The Transformer model (DistilGPT-2) behaved very differently. Its continuations were fluent, grammatically correct, and stayed on topic. It generated lines such as “The world is not yet clear” or longer, more expressive continuations like “The country and their world is up in the air and the United States is at a crossroads.” Even when randomness increased, the model maintained sentence structure, logical flow, and a sense of narrative direction. Instead of repeating words nonsensically, it used repetition only when it served a grammatical purpose. Overall, its responses sounded like complete thoughts rather than disconnected word fragments. The Transformer performs better because it processes language in a fundamentally more powerful way. Its self-attention mechanism allows it to look at the entire sequence of previous words—much more than the fixed two- or three-word window an N-gram model uses. This lets it understand long-range dependencies, syntax, and semantic relationships. On top of that, Transformers use dense vector embeddings, giving the model a nuanced understanding of word meaning and context. And because it is trained on a massive, diverse dataset, it can generalize even when it encounters unusual prompts or partial phrases. In contrast, the N-gram model is limited to shallow statistical patterns. Once its tiny memory is exhausted or the exact context doesn’t exist in training, it cannot infer grammar or meaning, so the output collapses into random, repetitive chains. The Transformer produces coherent, creative, and contextually meaningful text because it has the capacity to model real linguistic structure. The N-gram model simply doesn’t have the tools or context depth to compete.

3.5) **Beam Search Analysis**: Compare beam width of 2 vs 4 vs 8

- Does a larger beam width always produce better results?
- When might beam search be preferred over sampling?

Answer

When I compared beam widths of 2, 4, and 8, the results made it clear that increasing beam size doesn’t automatically improve generation quality. With the Word N-gram model, adjusting the beam width made almost no difference at all. Because the model only sees a very short context window, every beam tends to converge on the same high-probability next word, usually producing repetitive fragments like “The future of.” Even with a beam width of 8, the model didn’t discover any new or more meaningful continuations—it simply lacked the contextual information needed for

beam search to explore useful alternatives. The only thing that increased was the computation time, not the quality. With the Transformer model (DistilGPT-2), the effect was more noticeable, but still not strictly "better" as beam width increased. At a beam width of 2, the model produced a fluent and thoughtful continuation such as "The world is in the hands of the people, not the government." When I increased the width to 4 and 8, the outputs became more deterministic but also more generic, narrowing down to short, high-probability completions like "the United States" or "the U.S." These were correct and safe, but lost expressiveness. This is a typical beam-search behavior: larger beams often over-optimize for the most probable path, which leads to bland, compressed, and repetitive text rather than richer or more informative sentences. So a larger beam width does not always produce better results. In fact, going too wide can hurt creativity and reduce variety because the algorithm aggressively searches for the safest continuation rather than the most engaging one. A moderate beam size often works best, while an excessively large one can even push the model into loops or unhelpful deterministic phrasing. Beam search is generally preferred over sampling when a task needs reliable, consistent, and non-random outputs—things like machine translation, summarization, structured question answering, or any application where precision outweighs creativity. In those settings, the goal is to find the highest-quality single sequence, not a surprising or imaginative one. On the other hand, for open-ended or creative tasks—story writing, brainstorming, dialogue—sampling (top-k, top-p, etc.) produces more diverse, natural, and interesting text.

3.6) **Repetition Analysis**: Generate longer sequences (30+ tokens) with different strategies (different combinations of hyperparameters).

- Which strategies are most prone to repetition?
- How can you reduce repetition while maintaining quality?

Answer

When I generated longer sequences across different decoding strategies, repetition showed up in very predictable ways. The **Word N-gram model** suffered the most. With greedy decoding, it almost immediately fell into loops like "allow are allow attention algorithms attention," because it always picks the single most likely next word and has no real understanding of context. Even when I increased diversity using top-k sampling (k = 6–13), the model still produced repetitive fragments. A larger k made the output slightly more varied, but the underlying issue remained: the model only knows short fixed histories, so it runs out of meaningful options and keeps cycling through familiar words.

The **Transformer model (DistilGPT-2)** handled repetition far better thanks to its ability to look at long-range context. But even here, some strategies were more prone

to repetition. Beam search—especially with a large beam—tended to over-optimize and repeat short, safe phrases, producing deterministic outputs like "the U.S. economy" rather than exploring longer, richer continuations. Low-diversity settings such as small k or low p values also increased repetition because the model was forced to pick from a narrow set of high-probability words.

When I used more flexible sampling strategies, the repetitions dropped dramatically. For example, with top-k = 6 and top-p = 0.95, the model wrote longer, thoughtful lines like "the world is not yet clear, but the question of whether the world is a world without the human race." High-p sampling introduces controlled randomness, which helps the model escape repetitive loops while still staying mostly coherent.

Across all experiments, the strategies **most prone to repetition** were:

- greedy decoding (for both models),
- beam search with wide beams,
- low-diversity sampling (small k or low p).

To **reduce repetition while maintaining quality**, the most reliable approach is to use controlled sampling: moderate top-k, top-p around 0.8–0.95, and a slightly higher temperature. These settings widen the pool of possible next tokens just enough to break cycles without letting the model drift off-topic. For tasks where coherence matters but creativity is still important, this balance tends to produce the cleanest and most natural long-form text.

2️⃣ Reasoning Models

In this section we will first give you an overview of reasoning models and then provide you an interactive platform to test how reasoning and non-reasoning models behave differently.

What are reasoning models?

Reasoning models are LMs trained to "think first, answer second." They generate brief, intermediate text chains (often marked as `<think>…</think>`) before the final response, which helps their performance on multi-step problems. In your notebook, the reasoning model writes out its steps on grade-school math items, then gives the answer, whereas the base model jumps straight to the solution.

What are they used for?

- **Multi-step math & logic:** Show work, reduce arithmetic slips.
- **Causal/commonsense inference:** Make implicit steps explicit.

- **Structured outputs:** Stay within specific formats or constraints.

How are they trained?

- **Instruction-tuning on reasoning chains:** Train on examples with worked solutions ("rationales") plus final answers (e.g., GSM8K-style steps).
- **Reinforcement learning (RL) with LLM as a judge** for non-verifiable or free form text
- **RL with verifiable output** for math or coding

How do they generate text?

- **Prompting/templates for deliberation:** Chat templates and decoding settings encourage a short internal "scratchpad" before the final answer; your comparison runs hold temperature/top-p constant across the reasoning and base models.

Load model (phi4-mini)

Show code

Loading checkpoint shards: 100% 2/2 [00:02<00:00, 1.42s/it]

Loading checkpoint shards: 100% 2/2 [00:02<00:00, 1.39s/it]

```
# hidden layers in the base model
print("Number of hidden layers:", base_model.config.num_hidden_layers

# attention heads in the base model
print("Number of attention heads:", base_model.config.num_attention_h
```

```
Number of hidden layers: 32
Number of attention heads: 24
```

🔍 Q4: Inspect the loaded models

This question is auto-graded

Show code

Show Hint

4.1) How many hidden layers are there in the base model?

4.2) How many attention heads are there in the base model?

Submit Answers

Question 1: Correct! (32) Question 2: Correct! (24)

4.3) Code implementation

```
# Write some code to inspect the attributes of the loaded models here
# The variable names of the models are `reasoning_model` and `base_mo
# They have been loaded for you as HuggingFace Transformers models.

# YOUR CODE HERE

def model_inspect(model, name):
    print(f"\n{name} Information:")
    print("Model type:", model.config.model_type)
    print("Hidden layers:", model.config.num_hidden_layers)
    print("Vocab size:", model.config.vocab_size)
    print("Activation function:", model.config.hidden_act)
    print("Torch dtype:", model.config.torch_dtype)
    print("Initializer range:", model.config.initializer_range)
    print("Attention heads:", model.config.num_attention_heads)
    print("Hidden size:", model.config.hidden_size)
    print("Intermediate size:", model.config.intermediate_size)
    print("Max position embeddings:", model.config.max_position_embed


model_inspect(reasoning_model, "Reasoning Model")
model_inspect(base_model, "Base Model")
```

```
Reasoning Model Information:
Model type: phi3
Hidden layers: 32
Vocab size: 200064
Activation function: silu
Torch dtype: torch.bfloat16
Initializer range: 0.02
Attention heads: 24
Hidden size: 3072
Intermediate size: 8192
Max position embeddings: 131072

Base Model Information:
Model type: phi3
Hidden layers: 32
Vocab size: 200064
Activation function: silu
```

```
Torch dtype: torch.bfloat16
Initializer range: 0.02
Attention heads: 24
Hidden size: 3072
Intermediate size: 8192
Max position embeddings: 131072
```

Check model output in a real-world dataset

Note: this is not a question - please run these to set things up and to be sure the model is runnable for you, and get an idea of the format of each models' output

```
# Load data
from datasets import load_dataset

math_dataset = load_dataset("openai/gsm8k", "main")
```

```
questions = math_dataset['train']['question']
long_answers = math_dataset['train']['answer']

short_answers = []
for answer in long_answers:
    # Find the index of '####'
    index = answer.find('####')
    if index != -1:
        # Extract the text after '####' and strip any leading/trailin
        short_answer = answer[index + len('####'):].strip()
        short_answers.append(short_answer)
    else:
        # If '####' is not found, the short answer is the same as the
        short_answers.append(answer.strip())

# Prepare data for make_table function
cols = [questions, long_answers, short_answers]
colnames = ["Question", "Long Answer", "Short Answer"]

# Display the table
make_table(cols, colnames, title="GSM8K Dataset Examples")
```

GSM8K Dataset Examples

Question	Long Answer
Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did Natalia sell altogether in April and May?	Natalia sold 48/2 = May. Natalia sold 48+24 altogether in Apri #### 72
Weng earns $12 an hour for babysitting. Yesterday, she just did 50 minutes of babysitting. How much did she earn?	Weng earns 12/60 = minute. Working 50 minutes $<<0.2*50=10>>10. #### 10
Betty is saving money for a new wallet which costs $100. Betty has only half of the money she needs. Her parents decided to give her $15 for that purpose, and her grandparents twice as much as her parents. How much more money does Betty need to buy the wallet?	In the beginning, $<<100/2=50>>50. Betty's grandparen $<<15*2=30>>30. This means, Betty $<<100-50-30-15=5>> #### 5
Julie is reading a 120-page book. Yesterday, she was able to read 12 pages and today, she read twice as many pages as yesterday. If she wants to read half of the remaining pages tomorrow, how many pages should she read?	Maila read 12 x 2 = So she was able to <<12+24=36>>36 pag There are 120 - 36 left to be read. Since she wants to pages tomorrow, th <<84/2=42>>42 page #### 42
James writes a 3-page letter to 2 different friends twice a week. How many pages does he write a year?	He writes each fri week So he writes 6*2=<< That means he writ pages a year #### 624

Show code

Compare reasoning and base models

The goal of this section is to identify scenarios in which reasoning models outperform or underperform non-reasoning models and to inspect the model's reasoning.

You can use the interface below to experiment with some pre-set prompts and try out your own prompts.

Experiment with different prompts

Experiment with different prompts to observe the difference between reasoning and non-reasoning models. This might take 1-2 minutes to finish generating outputs.

Show code

🔍 Q5: Reasoning Behavior Analysis Questions

5.1) When did the reasoning model perform better, worse, or the same as the base model? Comment on the models' behaviors when using the pre-selected prompts.

Answer

When I compared the reasoning model with the base model across the pre-selected prompts, each one showed different strengths depending on the type of task.

For the Math / Logic Task (Betty's Wallet Problem), the reasoning model walked through the entire process step by step. It explained how half of 100 is 50, then added the contributions from her parents (15) and grandparents (30) to get 95, and finally subtracted 95 from 100 to find 5. It even took a moment to confirm that "half of the money she needs" was being interpreted correctly. The base model reached the exact same answer—5—but handled it in a much faster, compressed way, simply laying out the arithmetic (100÷2=50, 50+15+30=95, 100−95=5) without the step-by-step thinking. Both were correct, but the reasoning model emphasized clarity, while the base model emphasized efficiency.

For the Constrained Creative Task ("Describe Seattle Using Only Words Starting with S"), the difference between the two models was much more noticeable. The reasoning model got tangled in its own analysis. It kept checking every word, worrying about grammar, and revising repeatedly, but it never actually delivered a clean, fully constrained answer. The base model, on the other hand, produced a smooth, alliterative description—"Seattle shines spectacularly, stunningly serene, splendidly splendid …"—and successfully kept every word starting with "S." In this case, the base model clearly performed better because it focused on generating the output instead of overthinking the constraints.

For the Cause-and-Effect Task ("Rain and Wipers"), both models again ended up with the same correct conclusion. The reasoning model thoroughly examined both possible directions of causation, identified why wipers cannot cause rain, and concluded that the onset of rain (Event A) led the driver to turn on the wipers (Event B). The base model delivered the same result but in a short, direct explanation: rain caused the driver to turn on the wipers. The reasoning model was more explicit about its logic, while the base model simply presented the correct relationship.

The reasoning model tended to perform better when step-by-step analysis added value, worse when overthinking got in the way of creative output, and about the same

as the base model when the task was straightforward and didn't require extended reasoning.

5.2) Try your own prompt and comment on the behavior you observe. Be creative!

Answer

When I tried my own prompt—**"Give updates about GTA VI"**—the two models behaved very differently, and the contrast made their strengths really obvious.

The reasoning model spent most of its effort talking itself through the problem. It went into a long internal monologue about Rockstar Games, possible release years, rumors about engines, E3 announcements, leaked concept art, and even hypothetical beta tests. But it never actually *gave* me a straightforward update. Instead, it overanalyzed every angle—what it remembered, what it wasn't sure about, what might be rumors, what sources normally say—before finally summarizing everything in a very cautious, qualified way. The output felt extremely careful and reflective, but also slow and full of speculation-management rather than useful information.

The base model, in contrast, answered the prompt directly and cleanly. It simply said that as of October 2023, Rockstar had not officially announced anything about GTA VI, and encouraged checking official sources like Rockstar Newswire or gaming outlets. It didn't wander, didn't speculate, and didn't waste time. It delivered a short, factual, and practical response.

Comparing the two:

- The reasoning model **overthinks** in a situation where the user probably just wants a clear update. It tries to reason through rumors and possibilities, but ends up with a long, meandering thought process.
- The base model **sticks to what is known**, avoids unnecessary speculation, and gives a concise answer without hesitation.
- For this kind of prompt—where information is limited and mostly factual—the base model is more useful because it stays grounded and avoids spiraling into hypotheticals.
- The reasoning model is helpful when the task actually requires structured reasoning, but on simple information requests like this, the extra analysis gets in the way.

Here, using this prompt showed that the base model handles straightforward, fact-based questions better, while the reasoning model can accidentally "overthink" and drift into long internal explanations that don't add much value.

5.3) Read some of the reasoning chains provided by the reasoning model. Do you think the reasoning makes sense and helped the model reach the correct final

answer?

Answer

Yes, the reasoning chains generally make sense, and they clearly helped the model reach the correct final answers. In the Betty's wallet problem, the model walked through the arithmetic step by step—calculating half of the goal, adding the contributions from her parents and grandparents, and verifying the totals before concluding that she still needed five dollars. That careful breakdown made the reasoning transparent and supported the correct result. In the Seattle "S-only" description, the chain acted more like a planning space. The model checked whether each word followed the constraint, revised phrases when they didn't fit, and tried to keep the sentence coherent. You can see the model using the reasoning section to strategize around the constraint, even if it sometimes slows itself down by thinking too much. For the rain and wipers example, the reasoning was methodical. The model considered both directions of causation, ruled out the impossible one, and explained why the rain leads the driver to activate the wipers. The chain shows the model validating its conclusion rather than just giving an answer.